

ASSIGNMENT 9

PyTest Integration with Fixtures & Execution Setup

Student Name:	Roshni Ghosh Tagore
Enrollment No:	12023002028175
Course:	Python for Automation
Academic Year:	2023–2027
Department:	Computer Science and Engineering (Artificial Intelligence & Machine Learning)
Campus:	IEM Newtown

1. Problem Statement

Standard procedural automation scripts often mix setup, teardown, and assertion logic together, leading to redundant code and difficult maintenance. Furthermore, executing raw scripts directly makes detailed report logging cumbersome. To solve this, the existing Selenium automation setup needs to be refactored into PyTest, leveraging fixtures for browser lifecycle management and enabling HTML execution reporting.

2. Objective

- Convert standard linear Selenium scripts into modular PyTest test functions.
 - Implement `@pytest.fixture` in `conftest.py` for clean driver initialization and teardown (using `yield`).
 - Run the automated test suite directly via the PyTest CLI runner.
 - Assert UI element loading and page title verification for successful authentication.
-

3. Tools, Software, and Concepts Used

- **Programming Language:** Python 3.13.5
 - **Testing Framework:** PyTest 9.1.1
 - **Automation Tool:** Selenium WebDriver (`selenium.webdriver`)
 - **IDE/Editor:** Visual Studio Code (PowerShell Terminal)
 - **Core Concepts:** PyTest Fixtures, Teardown Pattern (`yield`), Web Element Locators, Assertions.
-

4. Source Code Implementation

Fixture Configuration File (conftest.py):

```
import pytest
from selenium import webdriver

@pytest.fixture
def driver():
    driver = webdriver.Chrome()
    driver.maximize_window()

    yield driver

    driver.quit()
```

Test Script File (test_login9.py):

```
def test_login(driver):
    driver.get("https://www.saucedemo.com/")

    driver.find_element("id", "user-name").send_keys("standard_user")
    driver.find_element("id", "password").send_keys("secret_sauce")
    driver.find_element("id", "login-button").click()

    assert "Swag Labs" in driver.title
```

5. Terminal Output Log

```
(.venv) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\seleniumassignment\Assignment 2nd wk> pytest
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\seleniumassignment\Assignment 2nd wk
plugins: anyio-4.13.0, html-4.2.0, metadata-3.1.1
collected 1 item

test_login9.py . [100%]

===== 1 passed in 8.51s =====
(.venv) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\seleniumassignment\Assignment 2nd wk>
```

6. Result, Observation, and Conclusion

Results: The PyTest test discovery automatically identified `test\_login9.py`. The driver fixture initialized Chrome, performed authentication, confirmed the page title contained 'Swag Labs', and safely closed the browser in 8.51s with 100% passing rate.

Observations: Moving setup/teardown code into `conftest.py` using `yield` ensures the browser quits cleanly regardless of whether assertions pass or fail. Integrating `pytest-html` plugin enables seamless reporting.

Conclusion: The integration of PyTest was successful, creating a cleaner modular automation structure with robust fixture lifecycle management.